

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192365045
Name	Shaik Mahaboob Basha

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I Shaik Mahaboob Basha (192365045) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Mahaboob Basha

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

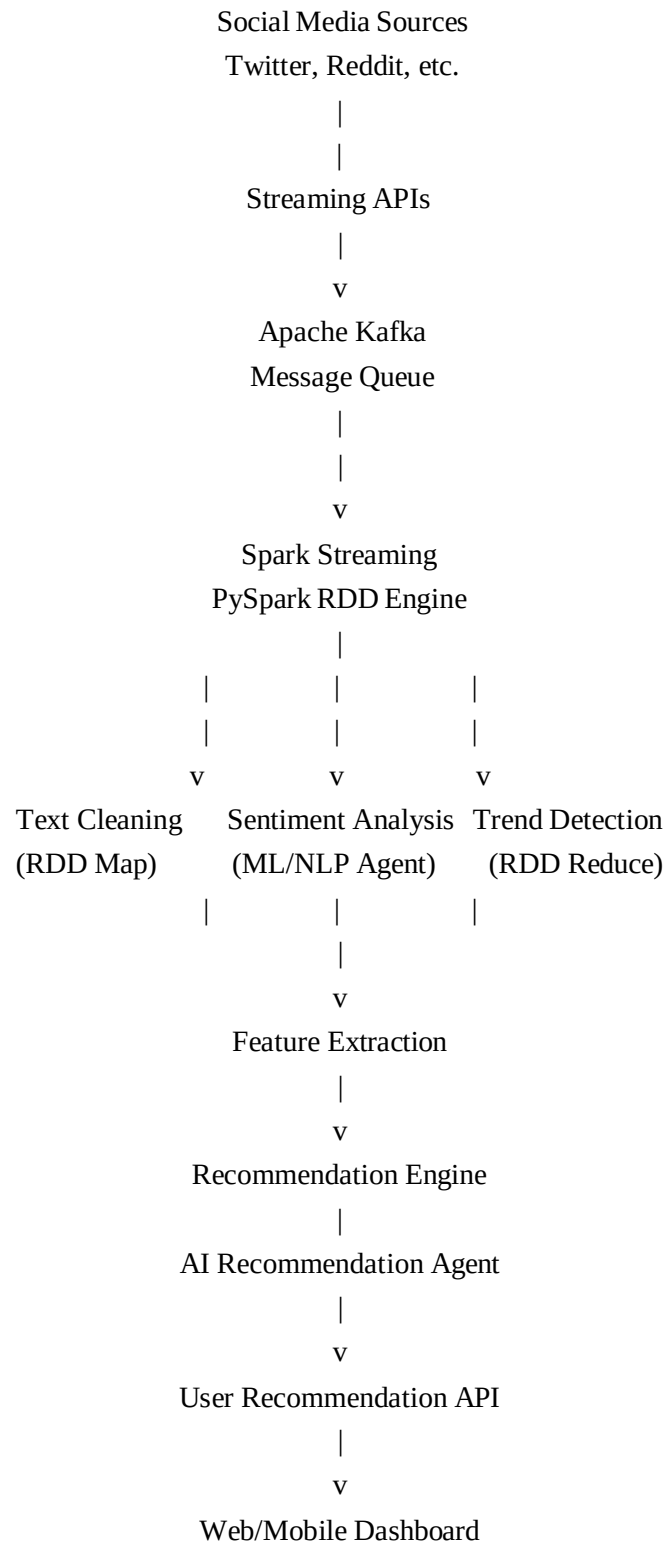
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1Q)

End-to-End System Design: AI-Based Real-Time Social Media Analytics and Recommendation System Using PySpark (RDD Operations)

1. Objective

Design a scalable AI-powered system that collects real-time social media data, processes it using PySpark RDD operations, performs sentiment and trend analysis, and generates personalized recommendations through intelligent AI agents.



stage	technology	function & data transformation
1. ingestion	kafka broker	buffers incoming json events: {user_id, post_id, text, timestamp, engagement_type, tags}.
2. stream parsing	pyspark dstream / rdds	consumes kafka partitions into micro-batches (e.g., 2–5s windows) as pyspark rdds.
3. preprocessing	pyspark mappartitions	strips urls, normalizes text, extracts hashtags, and filters low-quality spam.
4. feature aggregation	rdd reducebykey / aggregatebykey	computes rolling engagement scores, user-interaction graphs, and topic velocities.
5. ai inference	triton / onnx runtime + langchain	enriches rdd partitions with sentiment tags, intent classifications, and text embeddings.
6. agent synthesis	autonomous recommender agent	synthesizes user preferences, trending momentum, and collaborative filters.
7. serving	redis & vector store	exposes top-n recommendations via sub-50ms rest/grpc endpoints.

1. Social Media

Social media platforms generate large amounts of data such as posts, comments, likes, hashtags, and shares. This is the **input data** for the system.

2. Kafka

Apache Kafka collects the continuous stream of social media data and transfers it reliably to the processing system. It acts as a **message broker** between data sources and PySpark.

3. PySpark Streaming

PySpark Streaming receives the incoming data from Kafka and processes it in small batches or streams. It allows the system to analyze data **in near real time**.

4. RDDs

The incoming data is converted into **RDDs (Resilient Distributed Datasets)**. RDDs distribute the data across multiple machines so that processing can happen in parallel.

5. Data Cleaning

Unwanted information such as URLs, special characters, duplicate posts, and unnecessary words is removed. This produces clean data for further analysis.

6. Sentiment Analysis

The system analyzes the text to determine the user's sentiment:

- Positive

- Negative
- Neutral

7. Trend Detection

The system identifies frequently discussed topics, keywords, and hashtags. For example, if many users discuss "**Artificial Intelligence**", it can be identified as a trending topic.

8. User Profile

The system analyzes a user's posts, interests, likes, and interactions to create a profile. For example:

User: Rahul

Interests: AI, Python, Technology

Favorite Topics: Machine Learning

9. Recommendation AI Agent

The AI agent uses the user's profile together with current trends and past behavior to decide what content may be interesting to the user.

10. Personalized Recommendations

Finally, the system provides recommendations such as:

Recommended for Rahul:

- Machine Learning articles
- AI tutorials
- Python posts
- Trending AI discussions

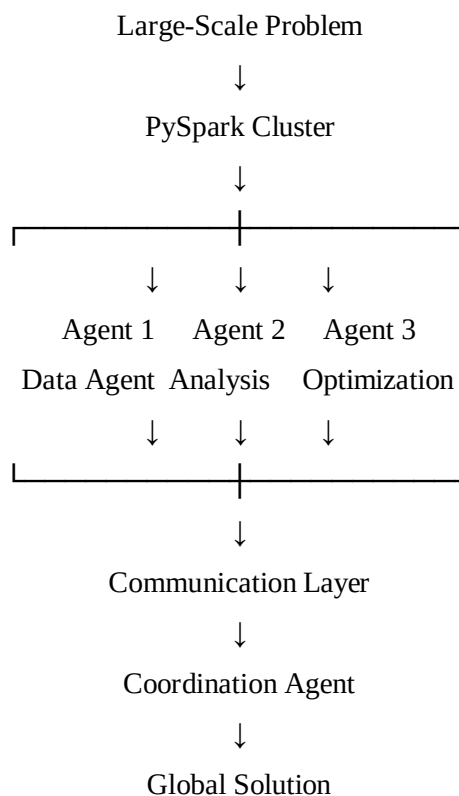
2Q) 2. Distributed Intelligent Agent System

Theory

A **Distributed Intelligent Agent System** consists of multiple AI agents that work together to solve a large-scale problem. Each agent performs a specific task and communicates with other agents to share results and make better decisions.

PySpark is used to distribute large amounts of data and computations across multiple machines. **RDDs** allow agents to process different parts of the problem in parallel.

Architecture



Working

1. Problem Input

A large optimization problem is given to the system, such as route optimization, resource allocation, scheduling, or recommendation optimization.

2. PySpark Distribution

PySpark divides the large dataset into smaller partitions using RDDs. These partitions can be processed simultaneously.

3. Multiple Intelligent Agents

Different agents work on different parts of the problem.

- **Data Agent** – collects and preprocesses data.
- **Analysis Agent** – analyzes the available data.
- **Optimization Agent** – searches for the best solution.
- **Evaluation Agent** – evaluates the quality of solutions.
- **Coordination Agent** – combines results from all agents.

4. Agent Collaboration

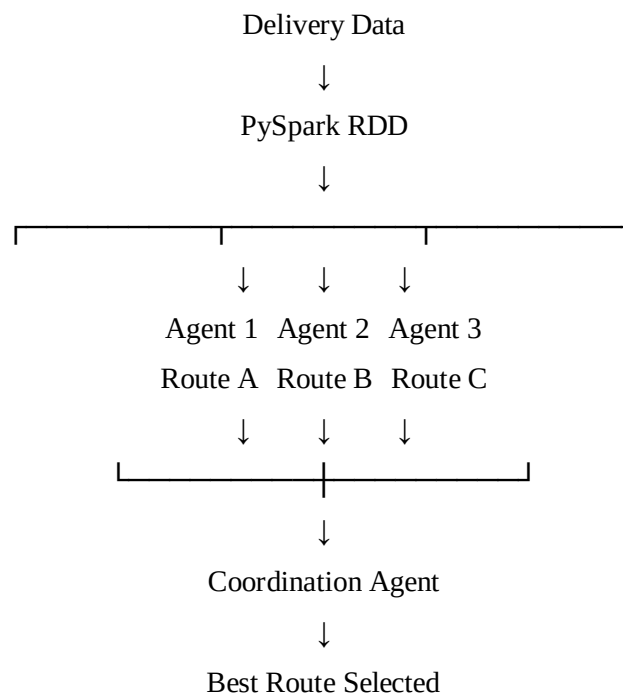
Agents exchange their intermediate results. For example, the Optimization Agents can send their best solutions to the Coordination Agent.

5. Global Optimization

The Coordination Agent compares the solutions and selects or combines the best results to produce the final solution.

Example

For **delivery route optimization**:



Advantages

- **Parallel processing** reduces execution time.
- **Scalable** for very large datasets.
- Agents can work **independently**.
- Agents can **share information and collaborate**.
- PySpark provides **fault tolerance** through distributed processing.
- Suitable for complex optimization problems such as scheduling, routing, resource allocation, and recommendation systems.

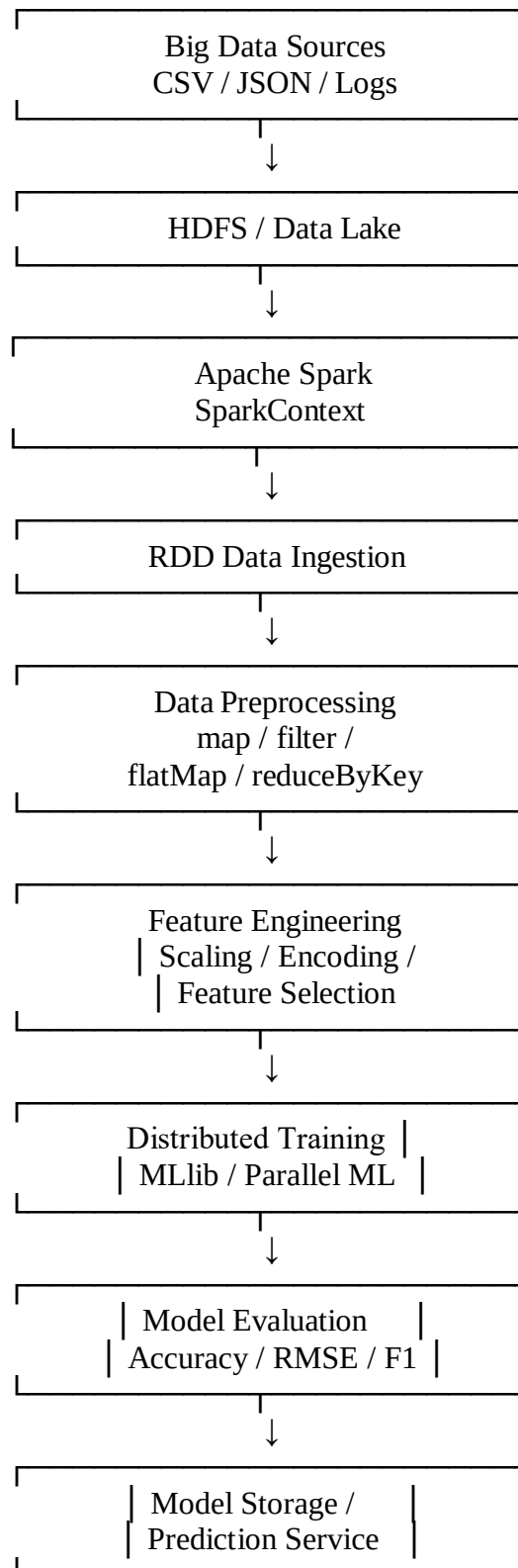
Key Benefits

Benefit	Explanation
Scalability	Can process very large datasets by adding more cluster nodes.
Parallelism	Multiple agents solve different tasks simultaneously.
Fault Tolerance	PySpark can recover lost computations using RDD lineage.
Collaboration	Agents share information to improve the overall solution.
Faster Processing	Distributed computation reduces the time required for large optimization problems.
Flexibility	New agents can be added for additional tasks without redesigning the entire system.
Better Decisions	Multiple agents evaluate different possibilities before selecting the final solution.

3Q)

Design a **complete distributed AI pipeline using Apache Spark RDD transformations** for large-scale data preprocessing, feature engineering, and model training while providing **scalability, parallel processing, and fault tolerance**.

Architecture



1. Data Ingestion

Large datasets are stored in **HDFS or a distributed data lake**. Spark creates an RDD from the input data.

```
from pyspark import SparkContext

sc = SparkContext("local[*]", "BigDataAI")

data = sc.textFile("hdfs://data/input.csv")
```

The input is automatically divided into **partitions**, allowing multiple Spark workers to process different portions simultaneously.

2. Data Preprocessing

RDD transformations are used to clean the data.

```
clean_data = (
    data
    .map(lambda x: x.split(","))
    .filter(lambda x: len(x) == 5)
    .filter(lambda x: x[0] != "")
)
```

Here:

- `map()` transforms every record.
- `filter()` removes invalid records.
- Processing happens in parallel across partitions.

Duplicate records can be removed using:

```
unique_data = clean_data.distinct()
```

3. Feature Engineering

The cleaned records are converted into feature vectors.

```
features = unique_data.map(
    lambda x: (
        float(x[1]),
        float(x[2]),
        float(x[3]),
        float(x[4])
    )
)
```

)

)

Additional transformations such as normalization, encoding, aggregation, and feature selection can be performed using RDD transformations.

For example:

```
keyed_data = unique_data.map(
```

```
    lambda x: (x[0], float(x[2]))
```

```
)
```

```
aggregated = keyed_data.reduceByKey(lambda a, b: a + b)
```

`reduceByKey()` performs distributed aggregation and minimizes unnecessary data transfer between nodes.

4. Distributed Model Training

After feature engineering, the data can be converted into a format suitable for **Spark MLlib**.

Example:

```
from pyspark.ml.linalg import Vectors
```

```
from pyspark.mllib.regression import LabeledPoint
```

```
training_data = features.map(
```

```
    lambda x: LabeledPoint(
```

```
        x[0],
```

```
        Vectors.dense(x[1:])
```

```
)
```

```
)
```

A distributed machine-learning algorithm such as **Logistic Regression, Linear Regression, Decision Tree, Random Forest, or K-Means** can then be trained using Spark's ML libraries.

5. Caching for Performance

If the same RDD is used repeatedly, it can be cached:

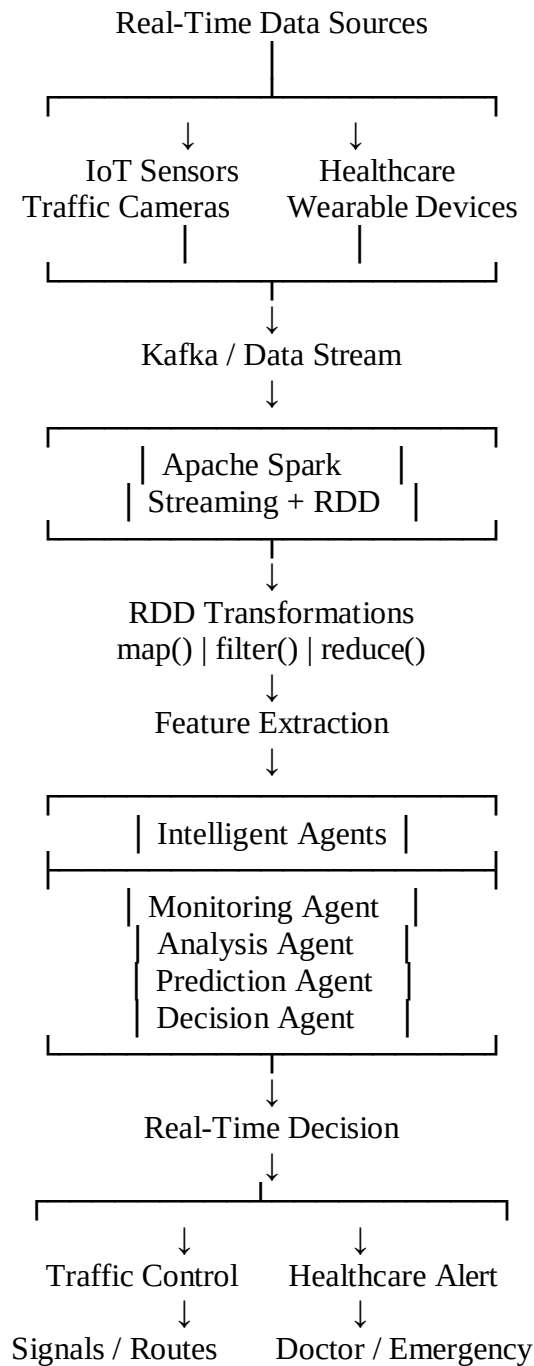
```
training_data.cache()
```

Caching avoids recomputing the RDD each time it is used, improving performance for iterative machine-learning algorithms.

4Q)

Construct an **AI-based real-time decision-making system** that combines **RDD-based streaming data processing** with **intelligent agents** to analyze incoming data, detect critical situations, and make decisions automatically. The system can be applied to **smart traffic management, healthcare monitoring, emergency response, and industrial automation**.

System Architecture



1. Real-Time Data Collection

Sensors continuously collect real-time information from the environment.

In **smart traffic**, sensors collect vehicle count, speed, road occupancy, accidents, signals, and weather.

In **healthcare**, wearable devices collect heart rate, blood pressure, oxygen level, temperature, and respiratory rate.

The collected data is sent to **Kafka**, which acts as a real-time data streaming platform.

2. RDD-Based Streaming Processing

Spark Streaming receives the continuous data and divides it into small time-based batches. Each batch is converted into an **RDD (Resilient Distributed Dataset)** for distributed processing. RDD operations clean and transform the incoming data before it reaches the AI agents. This allows large amounts of streaming data to be processed quickly and in parallel.

Stream Processing

Continuous Stream



Batch 1 → RDD 1 → Process

Batch 2 → RDD 2 → Process

Batch 3 → RDD 3 → Process

Batch 4 → RDD 4 → Process

The continuous data is divided into small batches. Each batch becomes an RDD and is processed independently. After processing, the results are sent to the intelligent-agent layer. This enables near real-time decision-making.

3. RDD Transformations

RDD transformations are used to clean, modify, and aggregate streaming data. `map()` transforms records, while `filter()` removes unwanted records. `reduceByKey()` combines values belonging to the same key. These operations are distributed across Spark workers for faster processing.

RDD Operation	Explanation
map()	Transforms each incoming record
filter()	Removes invalid or unwanted data
flatMap()	Converts one record into multiple records
reduceByKey()	Performs distributed aggregation
groupByKey()	Groups records with the same key
distinct()	Removes duplicate records
join()	Combines two datasets

4. Intelligent Agent Layer

Multiple AI agents work together to understand the streaming data.
Each agent has a specific responsibility such as monitoring, analysis, prediction, or decision-making.
The agents exchange their results with each other.
Finally, the Decision Agent selects the appropriate action.

Monitoring Agent

The Monitoring Agent continuously observes incoming sensor data.
It checks whether the data contains an abnormal or important event.
If everything is normal, monitoring continues.
If an abnormal condition is found, the event is sent to the Analysis Agent.

Analysis Agent

The Analysis Agent examines the detected event in detail.
It calculates values such as traffic density, average speed, or patient risk level.
It determines whether the situation is normal, moderate, or critical.
The result is passed to the Prediction Agent.

Vehicle Count = 250

Average Speed = 12 km/h

Road Occupancy = 92%

↓

Analysis Agent

↓

HIGH CONGESTION

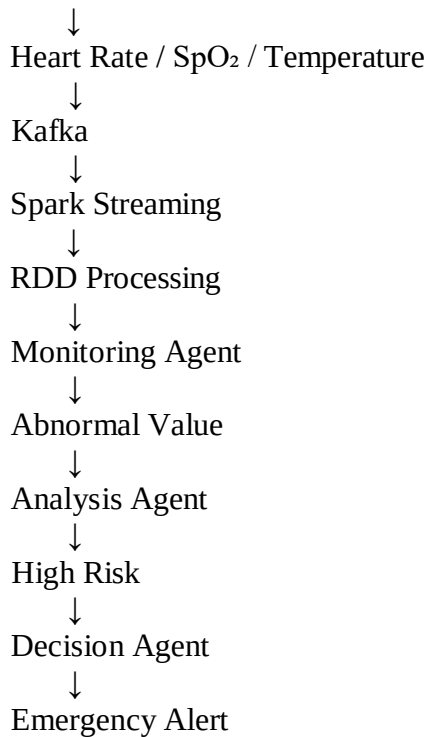
Prediction Agent

The Prediction Agent uses current and historical data to predict future conditions.
For example, it can predict whether traffic congestion will increase.
In healthcare, it can predict whether a patient's condition may become critical.
The prediction is sent to the Decision Agent.

The system checks traffic density and vehicle speed.
If traffic density is high and speed is low, congestion is detected.
The Decision Agent increases green-light duration.
It can also notify the traffic control center and suggest alternate routes.

Healthcare Decision System

Wearable devices continuously send patient health information.
Kafka receives the data and Spark Streaming processes it using RDDs.
AI agents analyze the patient's condition and identify abnormal values.
If the risk is high, the Decision Agent generates an alert for medical staff.
Patient Wearable



For example, abnormal oxygen level combined with abnormal heart rate can indicate a potentially serious situation.

The Analysis Agent evaluates these measurements together.

The Prediction Agent can estimate whether the condition is worsening.

The system can then alert a doctor or hospital according to validated clinical rules.

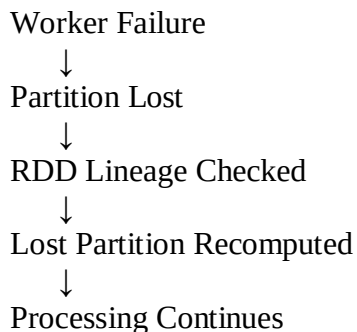
Fault Tolerance

Spark provides fault tolerance using **RDD lineage**.

RDD lineage records the sequence of transformations used to create an RDD.

If a worker fails and a partition is lost, Spark can recompute that partition.

Kafka can also retain streaming messages so that lost data can be processed again.



5Q)

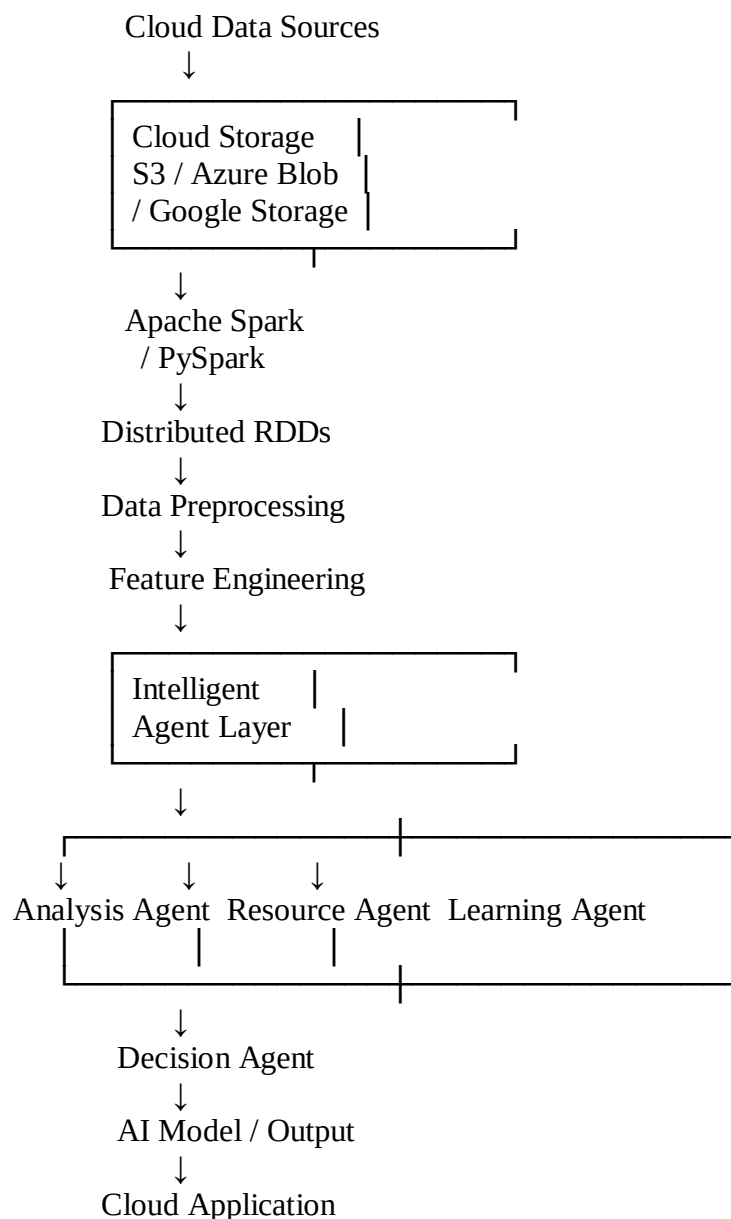
Design a **cloud-enabled distributed AI system** using **PySpark and intelligent agents** to process large-scale data.

The system uses cloud resources dynamically according to workload requirements.

AI agents analyze data, make decisions, and adapt the system based on changing conditions.

The architecture provides **scalability, fault tolerance, dynamic resource allocation, and adaptive learning**.

1. System Architecture



2. Cloud Data Collection

Data is collected from websites, IoT devices, applications, databases, and APIs.

The data is stored in cloud storage such as **Amazon S3, Azure Blob Storage, or Google Cloud Storage**.

Cloud storage allows very large datasets to be stored without depending on a single local machine.

PySpark can then access and process this distributed data.

3. Distributed Data Processing Using PySpark

PySpark distributes large datasets across multiple cloud worker nodes.

RDDs can be used to clean, transform, filter, and aggregate the data.

Multiple workers process different partitions simultaneously.

This makes the system faster and suitable for large-scale AI applications.

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("CloudDistributedAI") \
    .getOrCreate()
```

```
sc = spark.sparkContext
```

```
data = sc.textFile("s3://bucket/input/data.csv")
```

```
clean_data = data \
    .map(lambda x: x.split(",")) \
    .filter(lambda x: len(x) > 4)
```

```
print(clean_data.count())
```

Here, the data is loaded directly from cloud storage.

map() transforms the records, while filter() removes invalid records.

Spark distributes the RDD across available cloud workers.

Therefore, large datasets can be processed in parallel.

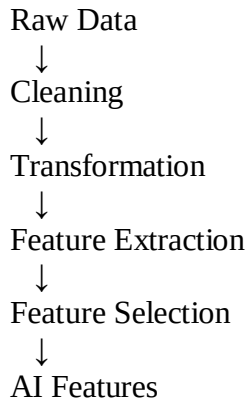
4. Feature Engineering

Feature engineering converts raw data into useful inputs for AI models.

PySpark can perform normalization, transformation, encoding, aggregation, and feature selection.

These operations can be distributed across multiple worker nodes.

The resulting features are passed to the AI model for training or prediction.



5. Intelligent Agent Architecture

Multiple intelligent agents perform different tasks in the cloud AI system.

Each agent specializes in monitoring, analysis, resource management, or learning.

Agents communicate with each other and share their results.

This makes the AI system more flexible and autonomous.

